

# Aula 2

Sensoriamento Remoto - PPEC



**“É preciso muito  
esforço para ser  
simples.”**

Clarice Lispector



# Configuração do Ambiente

Antes de treinar um modelo de deep learning para extrair pegadas de edificações ou classificar cobertura do solo a partir de imagens de satélite, você precisa de um ambiente de desenvolvimento funcional.

A stack de software GeoAI é mais complexa do que um projeto Python típico exigindo a interação de três ecossistemas de software:

- ❑ frameworks de deep learning (PyTorch, TensorFlow)
- ❑ bibliotecas C geoespaciais (GDAL, GEOS, PROJ)
- ❑ e pacotes de IA especializados (torchgeo, segment-geospatial, transformers)

# Requisitos de Hardware

## Requisitos Mínimos:

- CPU moderna multi-core (Intel i5/AMD Ryzen 5 ou equivalente)
- 8 GB de RAM (16 GB recomendado)
- Pelo menos 20 GB de espaço livre em disco para Python, pacotes e datasets de amostra
- Windows 10/11, macOS 14+ ou uma distribuição Linux recente (Ubuntu 24.04+)

# Requisitos de Hardware

## Requisitos Recomendados:

- GPU NVIDIA com 8 GB ou mais de VRAM (por exemplo, RTX 3070, RTX 4060 Ti ou superior)
- 16-32 GB de RAM do sistema
- SSD com pelo menos 50 GB de espaço livre
- Driver NVIDIA versão 590+ com suporte a CUDA 12.8 ou 13.0

## Alternativas em Nuvem:

- Google Colab: oferece acesso gratuito a GPUs NVIDIA T4.
- Kaggle: oferece acesso gratuito a GPUs NVIDIA T4 e P100.

# Instalando Drivers NVIDIA

O método mais simples no Windows é baixar o driver diretamente da NVIDIA:

1. Visite a página de Downloads de Drivers NVIDIA.
2. Selecione o tipo de produto, série e modelo da sua GPU (por exemplo, GeForce RTX 4090), depois escolha sua versão do Windows e clique em Search.
3. Baixe o instalador e execute-o. Escolha Express Installation para aceitar as configurações padrão.
4. Reinicie seu computador após a instalação ser concluída.

# Instalando Drivers NVIDIA

Após reiniciar, abra um terminal (Prompt de Comando ou PowerShell) e execute:

```
> nvidia-smi
```

|                            |                             |      |  |                        |      |                   |              |                    |         |         |     |
|----------------------------|-----------------------------|------|--|------------------------|------|-------------------|--------------|--------------------|---------|---------|-----|
| NVIDIA-SMI 573.44          |                             |      |  | Driver Version: 573.44 |      |                   |              | CUDA Version: 12.8 |         |         |     |
| GPU                        | Name                        |      |  | Driver-Model           |      | Bus-Id            | Disp.A       | Volatile           | Uncorr. | ECC     |     |
| Fan                        | Temp                        | Perf |  | Pwr:Usage/Cap          |      |                   | Memory-Usage | GPU-Util           | Compute | M.      |     |
|                            |                             |      |  |                        |      |                   |              |                    |         | MIG     | M.  |
| 0                          | NVIDIA RTX 2000 Ada Gene... |      |  | WDDM                   |      | 000000000:01:00.0 | Off          |                    |         |         | N/A |
| N/A                        | 48C                         | P3   |  | 13W / 44W              |      | 0MiB / 8188MiB    |              | 0%                 |         | Default | N/A |
| Processes:                 |                             |      |  |                        |      |                   |              |                    |         |         |     |
| GPU                        | GI                          | CI   |  | PID                    | Type | Process name      |              | GPU                | Memory  |         |     |
|                            | ID                          | ID   |  |                        |      |                   |              | Usage              |         |         |     |
| No running processes found |                             |      |  |                        |      |                   |              |                    |         |         |     |



# Instalação no Windows

1. Abra o PowerShell como administrador.
2. Baixe e execute o instalador do Miniconda:

```
curl https://repo.anaconda.com/miniconda/Miniconda3-latest-  
Windows-x86_64.exe -o .\miniconda.exe  
start /wait "" .\miniconda.exe /S  
del .\miniconda.exe
```

1. Feche o PowerShell. Abra o menu Iniciar e procure por Anaconda Prompt (Miniconda3).
2. Verifique a instalação:

```
conda --version  
python --version
```



# Criando um Ambiente Conda

Crie um ambiente dedicado chamado `geoai` com Python 3.13:

```
conda create -n geoai python=3.13 -y
```

Ative o ambiente:

```
conda activate geoai
```

Para desativar o ambiente e retornar ao ambiente base:

```
conda deactivate
```

# Instalando GeoAI e PyTorch (c/ GPU)

```
conda activate geoai
```

Execute o seguinte comando para instalar o PyTorch com suporte a GPU:

```
conda install -c conda-forge geoai segment-geospatial  
"pytorch*=cuda*"
```

Isso instala a versão mais recente do PyTorch com suporte a GPU. Para fixar uma versão específica, use o especificador de versão do `pytorch`. Por exemplo, para instalar o PyTorch 2.7.0 com suporte a CUDA 12.8, execute:

```
conda install -c conda-forge geoai segment-geospatial  
"pytorch=2.7.0=cuda128"
```

# Verificando o Acesso à GPU

Após a instalação, verifique se o PyTorch pode acessar sua GPU:

```
python -c "import torch; print(f'PyTorch {torch.__version__}');  
print(f'CUDA available: {torch.cuda.is_available()}');  
print(f'GPU: {torch.cuda.get_device_name(0)}' if  
torch.cuda.is_available() else 'CPU Only')"
```

Se o CUDA estiver disponível, você deve ver uma saída como:

```
PyTorch 2.13.0  
CUDA available: True  
GPU: NVIDIA RTX 2000 Ada Generation Laptop GPU
```

# Instalando uv como Gerenciador de Pacotes Alternativo

No Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex "
```

Usando pip:

```
pip install uv
```

Uma vez instalado, você pode usar `uv pip install` como um substituto direto para `pip install` dentro de qualquer ambiente virtual.

# Instalando com uv

Para criar um novo ambiente, execute:

```
uv venv --python 3.13
```

O ambiente virtual será criado no diretório `.venv`. Ative-o no Windows (PowerShell) com:

```
.venv\Scripts\Activate.ps1
```

Depois, instale os pacotes:

```
uv pip install geoai-py segment-geospatial jupyterlab
```

# Configurando o Jupyter

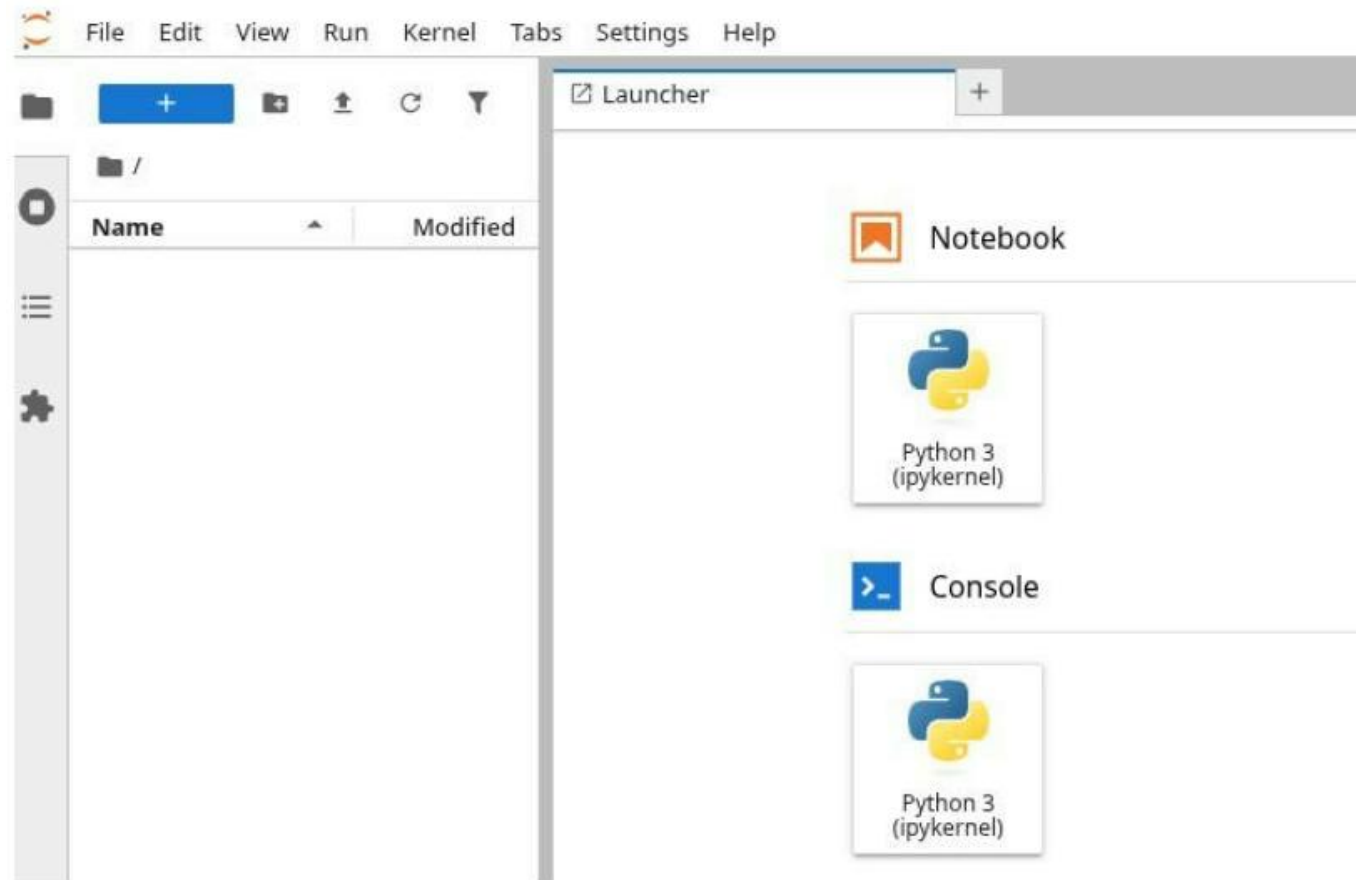
Se você seguiu os passos deste capítulo, o JupyterLab já deve estar instalado no seu ambiente `geoai`.

Inicie o JupyterLab a partir do seu terminal:

```
jupyter lab
```

Isso abre o JupyterLab no seu navegador. A partir daí, você pode criar novos notebooks clicando no tile Python 3 na seção Notebook do launcher.

# No JupyterLab





# JupyterLab

Cada notebook consiste em células que podem conter código ou texto formatado (Markdown). Crie uma nova célula de código e insira o seguinte código:

```
import torch
import geoai
print(torch.cuda.is_available())
print(geoai.__version__)
```

Execute a célula pressionando Shift + Enter . Você deve ver a saída na célula.

# Extensões Recomendadas no VS Code

Instale as seguintes extensões:

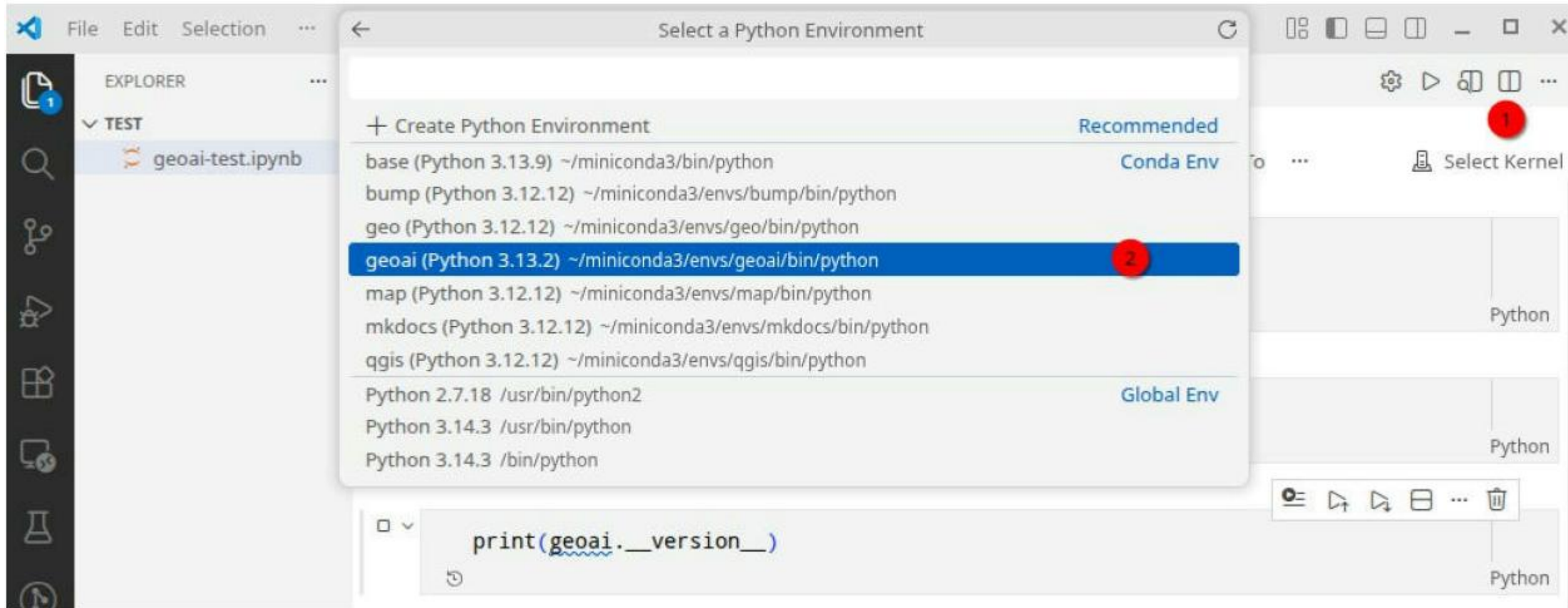
```
code --install-extension ms-python.python
```

```
code --install-extension ms-toolsai.jupyter
```

```
code --install-extension charliermarsh.ruff
```

1. Python (ms-python.python): Fornece IntelliSense, linting, depuração e gerenciamento de ambiente para Python
2. Jupyter (ms-toolsai.jupyter): Permite executar notebooks Jupyter diretamente dentro do VS Code
3. Ruff (charliermarsh.ruff): Um linter e formatador Python rápido que ajuda a escrever código limpo e consistente

# VS Code



# Fundamentos de Dados Geoespaciais

## O Que São Dados Raster?

Dados raster representam o mundo como uma grade regular de células, comumente chamadas de pixels.

Cada pixel armazena um ou mais valores numéricos que descrevem uma propriedade naquela localização, como reflectância de superfície, elevação ou temperatura.

# Propriedades chave

**Pixel (célula):** A menor unidade, armazenando um valor numérico.

**Banda:** Uma única camada de valores em toda a grade (por exemplo, reflectância no vermelho).

**Resolução:** A distância no terreno coberta por um pixel (por exemplo, 10 metros).

**Extensão:** A caixa delimitadora geográfica de toda a grade.

**Valor NoData:** Um valor sentinela indicando pixels ausentes ou inválidos.

# Imagens Multiespectrais e Hiperespectrais

O Sentinel-2, um dos satélites mais amplamente usados em GeoAI, fornece 13 bandas espectrais abrangendo comprimentos de onda visíveis, infravermelho próximo (NIR) e infravermelho de ondas curtas (SWIR). As bandas mais comumente usadas incluem:

- Banda 2 (Azul), Banda 3 (Verde), Banda 4 (Vermelho): Luz visível com resolução de 10 m
- Banda 8 (NIR): Infravermelho próximo com resolução de 10 m, sensível à saúde da vegetação
- Banda 11 (SWIR): Infravermelho de ondas curtas com resolução de 20 m, útil para umidade e geologia

# Índices espectrais derivados

NDVI (Índice de Vegetação por Diferença Normalizada):

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

NDWI (Índice de Água por Diferença Normalizada):

$$NDWI = \frac{Green - NIR}{Green + NIR}$$

# Resolução Espacial

A resolução espacial determina quanto detalhe do terreno um sensor pode capturar. Resolução mais fina revela objetos menores, mas tipicamente cobre menos área por cena e gera arquivos maiores.

| Satélite / Fonte | Resolução GSD(m) | Uso Típico                                         |
|------------------|------------------|----------------------------------------------------|
| Landsat 8/9      | 30 m             | Cobertura do solo regional, mudança de longo prazo |
| Sentinel-2       | 10 m             | Agricultura, mapeamento urbano                     |
| NAIP             | 0,3-1,0 m        | Imagens aéreas detalhadas dos EUA                  |
| Maxar, Airbus    | 0,3-0,5 m        | Deteção de edificações, infraestrutura             |



# Formatos Raster Comuns

**GeoTIFF** é o formato raster mais amplamente usado em trabalho geoespacial. Ele embute metadados geográficos (CRS, extensão, resolução) diretamente no cabeçalho do arquivo junto com os valores dos pixels. Praticamente toda ferramenta geoespacial pode ler e escrever arquivos GeoTIFF.

**Cloud Optimized GeoTIFF (COG)** é um GeoTIFF organizado com tiling interno e overviews para que o software possa ler apenas a porção necessária via HTTP sem baixar o arquivo inteiro. COGs são essenciais para fluxos de trabalho baseados em nuvem e são o formato padrão para a maioria dos arquivos modernos de dados de satélite.

**NetCDF** (Network Common Data Form) e **HDF5** (Hierarchical Data Format) são formatos de array multidimensional comumente usados para dados climáticos, meteorológicos e atmosféricos. Eles lidam com séries temporais e múltiplas variáveis em um único arquivo, tornando-os ideais para datasets como reanálise ERA5 ou produtos MODIS. No entanto, são menos comuns em tarefas típicas de classificação de imagens por GeoAI.

# Lendo Dados Raster com Python

A biblioteca `rasterio` é a ferramenta padrão para leitura de dados raster em Python. Ela fornece uma interface limpa para o GDAL, a Geospatial Data Abstraction Library subjacente.

```
import geoai
import rasterio

raster_url = "https://data.source.coop/opengeos/geoai/naip-train.tif"
raster_path = geoai.download_file(raster_url)
with rasterio.open(raster_path) as src:
    print(f"Shape: {src.height} x {src.width}")
    print(f"Bands: {src.count}")
    print(f"CRS: {src.crs}")
    print(f"Resolution: {src.res}")
    band1 = src.read(1) # Read the first band as a NumPy array
    print(f"Band 1 dtype: {band1.dtype}")
```

# Lendo Dados Raster com Python

Ler uma janela específica (subconjunto) de um raster grande é simples e frequentemente essencial ao trabalhar com cenas grandes:

```
clip_raster_path = "naip_clip.tif"
geoai.clip_raster_by_bbox(
    raster_path,
    clip_raster_path,
    bbox=(0, 0, 500, 500),
    bands=[1, 2, 3],
    bbox_type="pixel",
)
geoai.view_image(clip_raster_path)
```

# Dados Vetoriais

## O Que São Dados Vetoriais?

Enquanto dados raster se destacam na representação de fenômenos contínuos como reflectância, elevação e temperatura, muitas feições geográficas são melhor descritas como objetos discretos com limites bem definidos. Uma edificação tem bordas. Uma estrada tem uma linha central. Um lago tem uma margem.

Dados vetoriais capturam essas feições usando primitivos geométricos, tornando-os o formato natural para representar estruturas construídas pelo homem, limites administrativos e muitos rótulos de anotação em fluxos de trabalho de GeoAI.

# Formatos Vetoriais Comuns

**Shapefile** é um formato legado desenvolvido pela Esri no início dos anos 1990.

**GeoJSON** armazena feições vetoriais como texto JSON simples, tornando-o legível por humanos e fácil de usar em aplicações web.

**GeoPackage** é um formato moderno baseado em SQLite que suporta múltiplas camadas, nomes longos de campos e arquivos grandes em um único contêiner portátil.

**GeoParquet** estende o formato colunar Apache Parquet com metadados geoespaciais.

# Sistemas de Referência de Coordenadas

**CRS Geográfico** usa coordenadas angulares (longitude e latitude) em um modelo elipsoidal da Terra.

As coordenadas são em graus, e a distância real representada por um grau varia com a latitude. WGS 84 (EPSG:4326) é o CRS geográfico mais comum.

**CRS Projetado** transforma a superfície curva da Terra em um plano usando uma projeção matemática.

As coordenadas são em unidades lineares (tipicamente metros), o que torna os cálculos de distância e área mais fáceis. Exemplos incluem zonas UTM e Web Mercator (EPSG:3857).

# Formatos de Anotação para Deep Learning

Por Que Formatos de Anotação Importam ?

Modelos de deep learning precisam de dados de treinamento rotulados, e o formato desses rótulos deve corresponder ao que o framework de treinamento espera. Modelos de detecção de objetos tipicamente consomem anotações de caixas delimitadoras no formato COCO ou YOLO, enquanto modelos de segmentação semântica precisam de imagens de máscara em nível de pixel. Usar o formato errado, ou converter entre formatos incorretamente, é uma fonte comum de erros em projetos de GeoAI.

# Formato COCO

O formato COCO (Common Objects in Context) armazena anotações como um único arquivo JSON com três seções principais: *images* (metadados para cada imagem), *annotations* (caixas delimitadoras, polígonos de segmentação e IDs de categorias) e *categories* (definições de classes). Ele suporta tanto caixas delimitadoras quanto máscaras de polígonos, tornando-o versátil para tarefas de detecção de objetos e segmentação de instâncias.

As caixas delimitadoras COCO usam o formato [x\_min, y\_min, width, height] em coordenadas de pixel.



# Formato COCO

```
{
  "images": [{"id": 1, "file_name": "tile_001.png", "width": 512, "height":
512}],
  "annotations": [{"id": 1, "image_id": 1, "category_id": 1,
                    "bbox": [100, 120, 50, 40], "area": 2000}],
  "categories": [{"id": 1, "name": "building"}]
}
```

# Formato YOLO

As anotações YOLO usam um arquivo de texto por imagem, com cada linha representando um único objeto: `class_id center_x center_y width height` . Todas as coordenadas são normalizadas para o intervalo `[0, 1]` relativo às dimensões da imagem, tornando o formato compacto e fácil de analisar.

```
0 0.45 0.52 0.10 0.08
```

```
0 0.72 0.31 0.12 0.09
```

Cada linha acima descreve uma edificação (classe 0) com sua posição central e tamanho como frações da largura e altura da imagem.

# Formato Pascal VOC

O formato Pascal VOC usa um arquivo XML por imagem. Cada arquivo lista caixas delimitadoras com coordenadas de canto ( `xmin` , `ymin` , `xmax` , `ymax` ) e rótulos de classe. Embora menos comum em pesquisas recentes, permanece suportado por muitos frameworks de detecção e datasets legados.

# Máscaras Raster

Para segmentação semântica, rótulos são armazenados como imagens raster onde cada valor de pixel representa um ID de classe. Uma máscara de segmentação de edificações pode usar 0 para fundo e 1 para pixels de edificação. Quando o georreferenciamento importa, essas máscaras são frequentemente salvas como arquivos GeoTIFF de banda única para que preservem o mesmo CRS e extensão espacial que as imagens de entrada.

Máscaras raster são o formato de anotação mais natural para segmentação geoespacial porque mantêm o georreferenciamento. Quando você divide a imagem de entrada em tiles, você divide a máscara de forma idêntica, garantindo alinhamento perfeito.

# Escolhendo o Formato Certo

| Tarefa de IA              | Formato Recomendado        | Tipo de Saída                 |
|---------------------------|----------------------------|-------------------------------|
| Detecção de objetos       | COCO, YOLO, Pascal VOC     | Caixas delimitadoras          |
| Segmentação de instâncias | COCO (com polígonos)       | Máscaras por objeto           |
| Segmentação semântica     | Máscaras raster (GeoTIFF)  | Classes em nível de pixel     |
| Detecção de mudanças      | Máscaras raster pareadas   | Mapas de classes antes/depois |
| Classificação de cena     | Estrutura de pastas ou CSV | Rótulos em nível de imagem    |

# Tiling de Imagens e Chips

## Por Que o Tiling é Necessário?

Imagens de satélite são grandes. Um único tile do Sentinel-2 cobre 100 x 100 km com 10.980 x 10.980 pixels.

Uma cena NAIP pode exceder 10.000 x 10.000 pixels em resolução sub-métrica. Modelos de deep learning, no entanto, tipicamente aceitam entradas de tamanho fixo (por exemplo, 256 x 256 ou 512 x 512 pixels), e limites de memória GPU restringem ainda mais os tamanhos de lote. A solução é cortar imagens grandes em patches menores e uniformes, geralmente chamados de tiles ou chips.

# Considerações sobre Tamanho do Tile

Tamanhos de tile comuns para GeoAI incluem:

**256 x 256:** Padrão para muitas arquiteturas de segmentação, equilibra contexto e memória

**512 x 512:** Fornece mais contexto espacial por tile, útil para objetos maiores

**1024 x 1024:** Usado quando objetos são muito grandes ou o contexto é crítico

Sobreposição entre tiles adjacentes (por exemplo, 32 ou 64 pixels) ajuda a evitar cortar objetos nos limites dos tiles. Durante a inferência, previsões de regiões sobrepostas são mescladas para produzir mapas de saída contínuos.

# Mantendo o Contexto Geoespacial

Diferentemente de imagens naturais, tiles de satélite devem preservar suas coordenadas geográficas. Quando você extrai um chip de um GeoTIFF maior, o chip precisa de sua própria transformação afim válida e CRS para que as previsões possam ser mapeadas de volta para localizações do mundo real. Durante o aprendizado supervisionado, as imagens e seus rótulos também devem ser divididos em tiles com a mesma grade e sobreposição para permanecerem alinhados. A função `leafmap.split_raster()` lida com o lado raster desse fluxo de trabalho automaticamente.



# Mantendo o Contexto Geoespacial

Além de formatos de arquivo individuais, a comunidade geoespacial desenvolveu padrões de catálogo para organizar e descobrir grandes coleções de dados. A especificação SpatioTemporal Asset Catalog (STAC) define uma maneira comum de descrever e pesquisar arquivos de imagens de satélite hospedados por provedores como Microsoft Planetary Computer e NASA Earthdata.